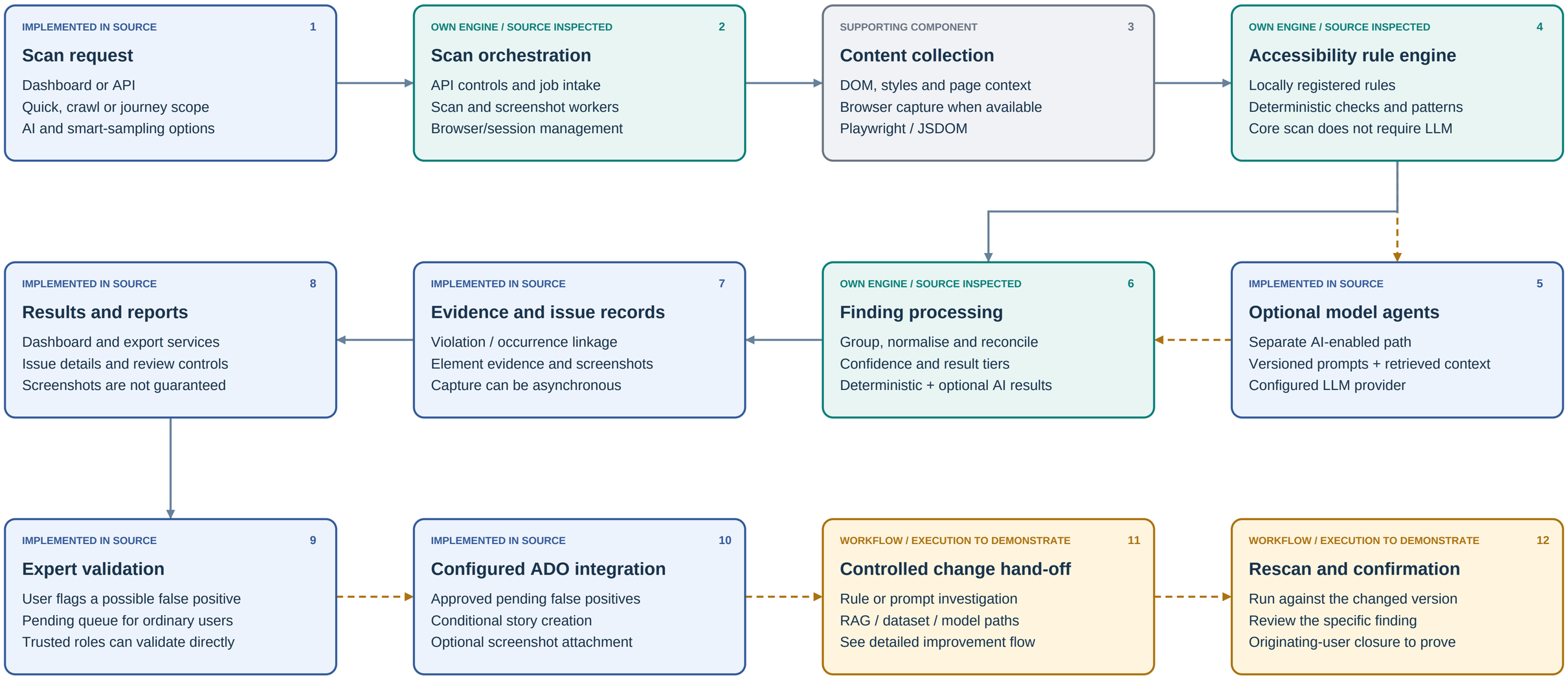
IntelliA11y | Scan and assurance architecture

Custom accessibility analysis with optional model interpretation and expert review



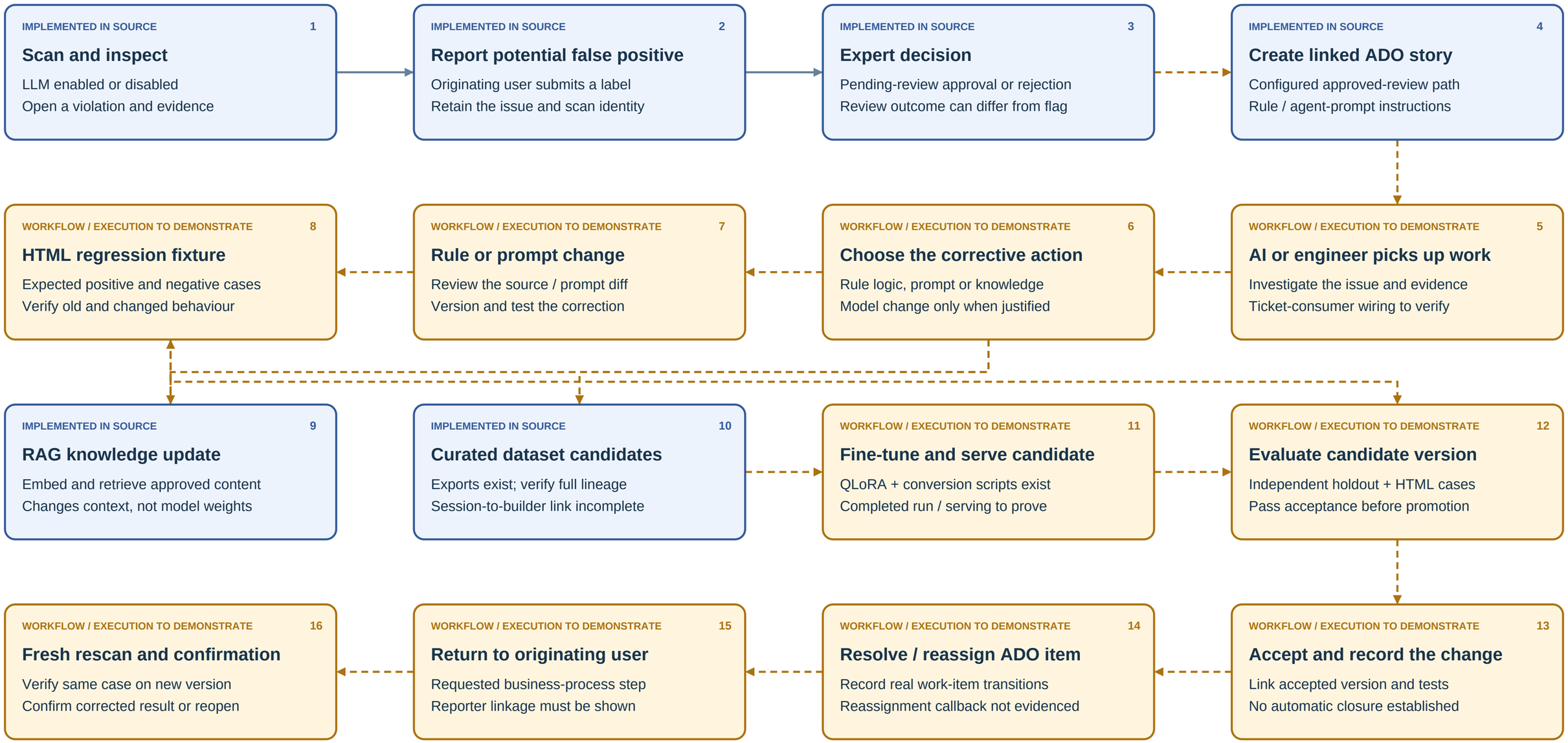
API stores: PostgreSQL / vector context and Redis. Separate CLI: own rules with Puppeteer / JSDOM; outside this API workflow.

Own-engine boundary: primary API/CLI paths execute local rules. axe appears in a separate comparison script, not the inspected core scan path.

Ticket creation does not prove automatic code changes, fine-tuning, deployment or final user confirmation. Source detail: intellia11y_analysis.txt (F01-F27).

IntelliA11y | Expert-governed improvement loop

The requested end-to-end process, with implemented entry points and explicit execution hand-offs



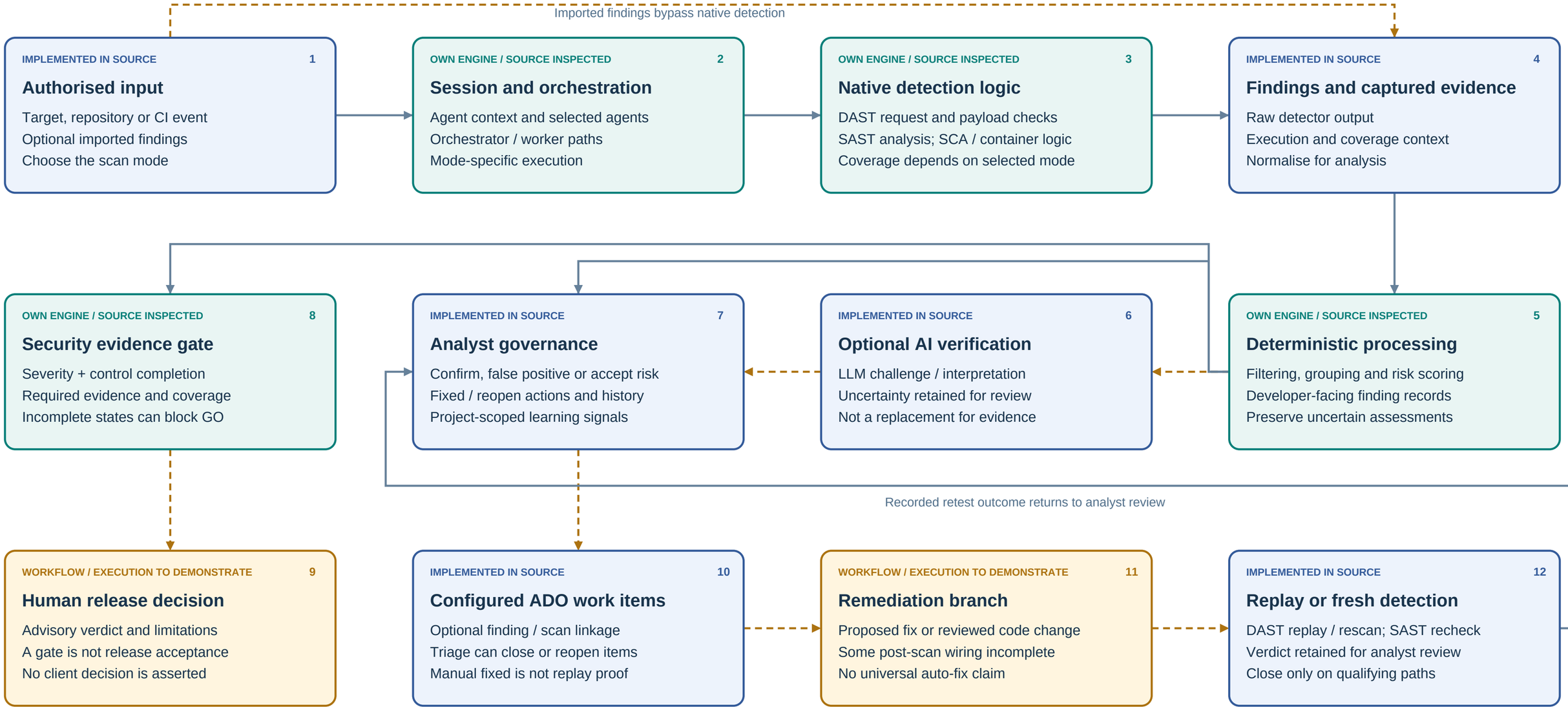
Rule edits, prompt edits, RAG updates and fine-tuning are separate mechanisms. A false-positive review need not trigger all four.

Solid links show inspected workflow connections. Dashed links show intended/manual or external execution; they are not claimed autonomous.

Dataset overlap must be resolved before independent uplift claims. A generated HTML fixture is a test case, not evidence of a client outcome.

IntelliSec | Native detection and governed remediation

Mode-specific detector logic, evidence-led triage and explicit retest branches



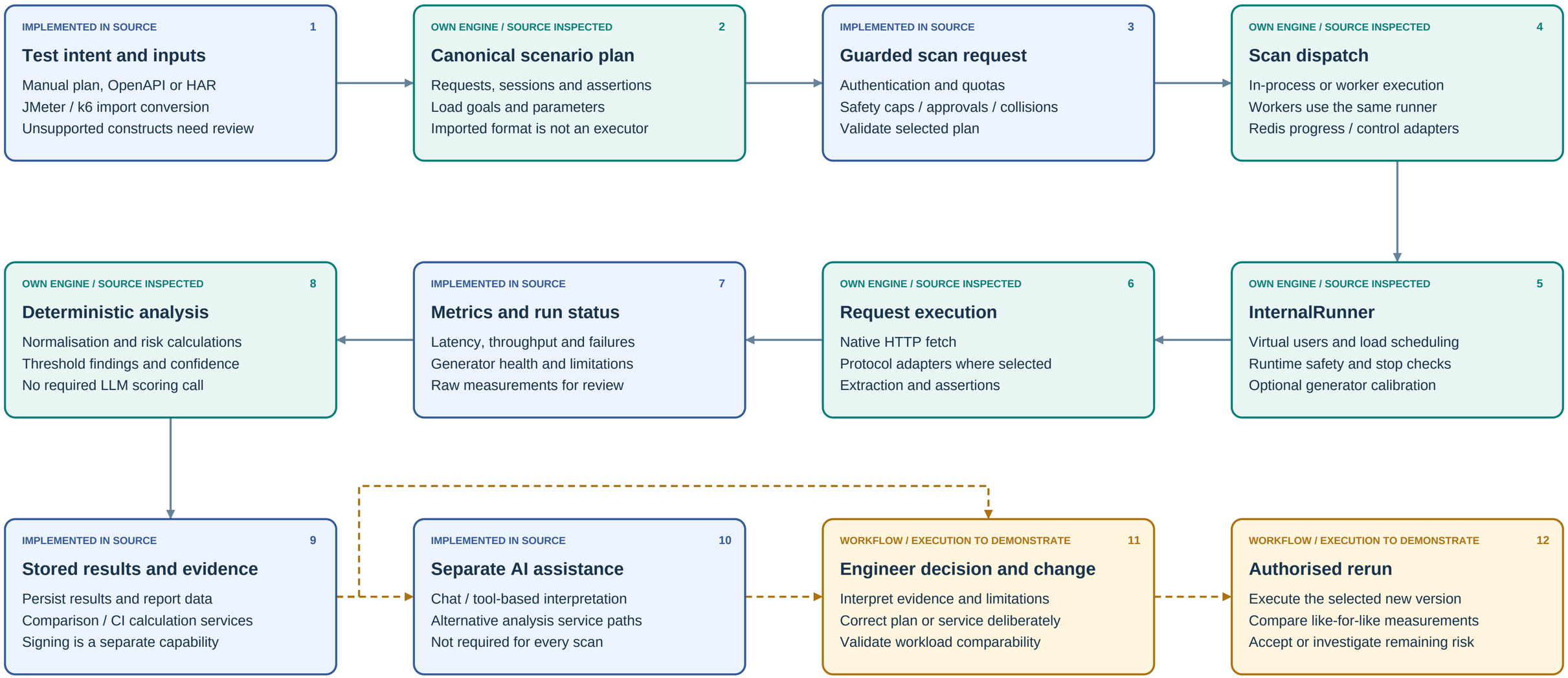
Improvement mechanisms: scoped suppressions and rule candidates; prompt versions, A/B mechanisms and training-data suggestions.

Own-engine boundary: native detectors coexist with optional imports, advisory feeds, policy tools and utility integrations. No zero-dependency claim.

Retest-backed and manual closure paths differ. Do not equate every fixed status with a verified fix. Source detail: intellisec_analysis.txt.

Inteliperf | Native workload and analysis architecture

Deterministic execution and measurement with separate AI assistance



Prototype boundary: auto-remediation actions and rollback are simulated. Mock regional runners are not proof of live distributed execution.

Own-engine boundary: the inspected core invokes InternalRunner and native requests, not k6/JMeter executables. AI and imports are separate components.

Calibration can estimate capacity; scoring and comparison semantics need runtime validation. Separate from client PerfOps / Perf Insights. See inteliperf_analysis.txt.